Dynamic Networks

EdgeBasedModels.jl Vignette 06

Simon Frost

2026-05-14

Table of contents

Introduction . . . 1
Setup . . . 2
Static SIR Baseline . . . 2
Dynamic SIR Model . . . 5
Effect of Edge Dynamics . . . 8
Inspecting Edge States . . . 11
Dynamic SEIR . . . 12
Simulation validation . . . 14

Introduction

Real contact networks are not static — edges form and break over time as individuals change partners, move between workplaces, or alter social behaviour during an epidemic. The `DynamicConfigurationModel` in EdgeBasedModels.jl extends the static EBCM by introducing dormant edge stubs that capture this turnover.

In the dynamic model, active edges break at rate η_2 (becoming dormant) and dormant edges form at rate η_1 (becoming active). This yields an additional state variable φ_D for dormant edges. The key equations are:

$$\frac{d\theta}{dt} = -\underbrace{\beta\varphi_I}_{\text{transmission}} \underbrace{-\eta_2(\varphi_S + \varphi_I + \varphi_R)}_{\text{edge breaking}} + \underbrace{\eta_1\varphi_D}_{\text{edge forming}}$$

$$\frac{d\varphi_D}{dt} = \eta_2(\varphi_S + \varphi_I + \varphi_R) - \eta_1\varphi_D$$

When η_1 and η_2 are both zero, the model reduces to the static EBCM. When edge turnover is fast, the network approaches mean-field mixing. This framework can model partnership dynamics, workplace contacts, or serosorting — where contact patterns change based on disease status.

Setup

```
using EdgeBasedModels
using ModelingToolkit
using Symbolics
using OrdinaryDiffEq
using Plots
```

We define all symbolic parameters used throughout this vignette.

```
@parameters β γ κ η₁ η₂ σ
pgf = poisson_pgf(κ)
```

```
DegreePGF(z, exp((-1 + z)*κ))
```

We also define a helper function to extract population-level trajectories from an ODE solution, given a Poisson PGF.

```
function extract_SIR(sol, result, κ_val)
    ψ(x) = exp(κ_val * (x - 1))
    θ_vals = compartment(sol, result, :θ)
    S = ψ.(θ_vals)
    R = compartment(sol, result, :R)
    I = 1.0 .- S .- R
    return S, I, R
end
```

```
extract_SIR (generic function with 1 method)
```

Static SIR Baseline

Before introducing edge dynamics, we build a standard static SIR on a Poisson network with mean degree $\kappa = 5$.

> **Note**
>
> $R_0 = 2$ anchor. We use $\gamma = 0.25$, $\kappa = 5$, $T = 2/5$, and per-edge $\beta = 1/6$ with 1% initial infection. Dynamic variants reuse these disease parameters so changes reflect edge turnover.

```
static_result = build_sir(pgf, β, γ; form=:expanded)
γ_val = 0.25
R0_target = 2.0
seed_fraction = 0.001
κ_val = 5.0
T_val = R0_target / κ_val
β_val = T_val * γ_val / (1 - T_val)
```

```
0.16666666666666669
```

```
ic_static = default_initial_conditions(static_result; seed_fraction = seed_fraction)
p_static = Dict(β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val)
tspan = (0.0, 40.0)
prob_static = ODEProblem(static_result.system, merge(ic_static, p_static), tspan)
sol_static = solve(prob_static, Tsit5())
```

```
retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 20-element Vector{Float64}:
  0.0
  0.13476445163359485
  0.6737507704907311
  1.508860712739542
  2.4775296157952225
  3.6784359626940017
  5.055215574371413
  6.632943657585046
  8.408748854385125
 10.435590308139949
 12.868964007508328
 15.497599661290707
 18.4434571509689
 21.49481000560798
 24.928074508150978
 28.766432614033665
 32.31055129636298
```

```
 35.97160804114679
 39.62651311029758
 40.0
u: 20-element Vector{Vector{Float64}}:
 [0.0, 0.001, 0.0, 0.0010000000000000009, 1.0]
 [3.503419723055881e-5, 0.0010803534267972812, 3.465293944645891e-5, 0.0010576327249504094,
 [0.000204246372172012, 0.0014428571624426271, 0.0001943883501237785 3, 0.001323122951075009
 [0.0005772203634075918, 0.0021679712041615747, 0.0005245398513846509, 0.0018709584819280832
 [0.0012373798382311913, 0.0033588991825453753, 0.0010819121953544119, 0.0027930920285192157
 [0.0025597465485769602, 0.005629588344056867, 0.0021667652159541376, 0.004578059532710266,
 [0.005178247119923547, 0.009976765255827845, 0.004281155538986428, 0.008019753144107347, 0.
 [0.010687675080550624, 0.01884155612702293, 0.008692039791499942, 0.015042498221740315, 0.9
 [0.02272966074970463, 0.037297311719169134, 0.0182710216883117, 0.029575269655020924, 0.987
 [0.05042422414223807, 0.07554705459658571, 0.040092140149284194, 0.05915104515668345, 0.973
 [0.1169717446022091, 0.146380384383823, 0.09139284412203011, 0.11103072211598193, 0.9390714
 [0.23775243788223324, 0.21412724402248773, 0.18007432217375224, 0.15175581161513388, 0.8799
 [0.40188953517593745, 0.21808242380833207, 0.2899451271852463, 0.13673008034219236, 0.80670
 [0.5497349639438021, 0.1650286731656013, 0.376029956663388, 0.08804704267042325, 0.7493133 6
 [0.662504270952474, 0.10015346965350906, 0.43117295213638407, 0.044036153712009536, 0.71255
 [0.7329384131112342, 0.05105182359262976, 0.4594316194651154, 0.018270870928671547, 0.69371
 [0.7658650263926947, 0.02584262848213048, 0.47034706238570917, 0.007795884231976461, 0.6864
 [0.7826230079742404, 0.012333438336791252, 0.4750634674449842, 0.003184000569391273, 0.6832
 [0.7905173455661024, 0.005745795129706234, 0.47698149178005383, 0.0012939877290521818, 0.68
 [0.7910331565859934, 0.005308351221185 58, 0.4770969100547465, 0.0011799910492679764, 0.6819
```

```
S_s, I_s, R_s = extract_SIR(sol_static, static_result, 5.0)
plot(sol_static.t, S_s, label="S", lw=2, color=:blue)
plot!(sol_static.t, I_s, label="I", lw=2, color=:red)
plot!(sol_static.t, R_s, label="R", lw=2, color=:green)
xlabel!("Time")
ylabel!("Fraction of population")
title!("Static SIR (κ=5, R₀=2, γ=0.25)")
```

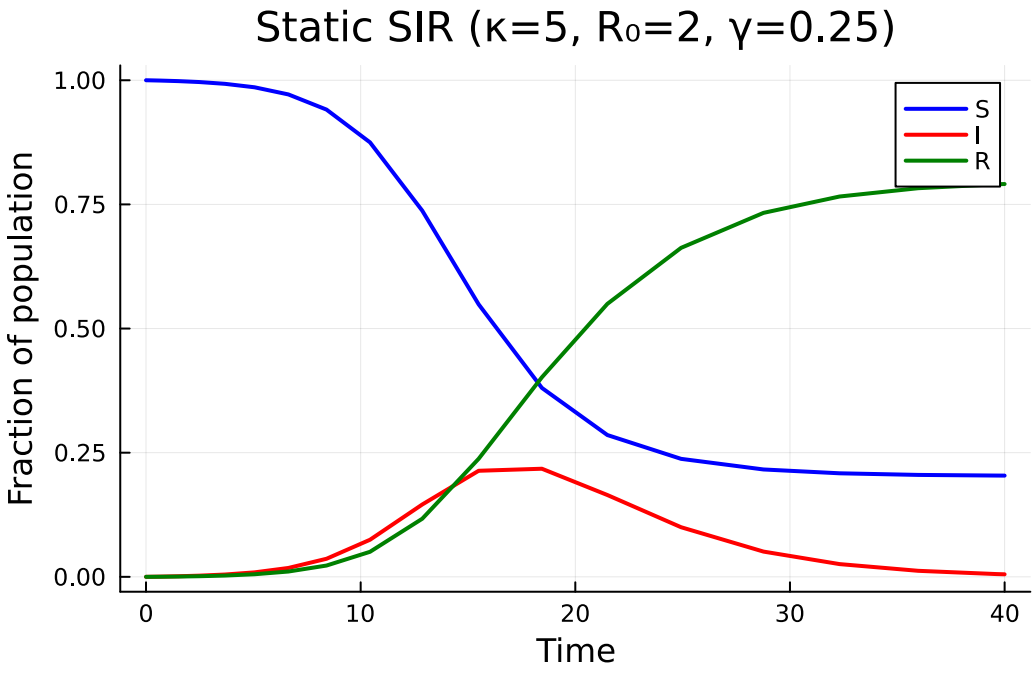

Figure 1: SIR epidemic on a static Poisson network with mean degree $\kappa = 5$.

Dynamic SIR Model

Now we build an SIR model on a dynamic network where edges break at rate $\eta_2 = 0.3$ and form at rate $\eta_1 = 0.5$. At equilibrium (before the epidemic), the fraction of dormant edges would be $\eta_2/\eta_1 = 0.6$. Since we start with all edges active ($\varphi_D(0) = 0$), the dormant population builds up as edges break.

```
progression = DiseaseProgression(
    [DiseaseStage(:I; transmission_rate=β), DiseaseStage(:R)],
    [DiseaseTransition(:I, :R, γ)];
    entry=:I
)
dyn_model = DynamicConfigurationModel(pgf, progression, η₁, η₂)
dyn_result = build_edge_system(dyn_model)
```

```
EdgeModelSystem(Model dynamic_ebm:
Equations (6):
  6 standard: see equations(dynamic_ebm)
Unknowns (6): see unknowns(dynamic_ebm)
  pop_R(t)
```

```
  pop_I(t)
  φ_R(t)
  φ_I(t)
  ⋯
Parameters (6): see parameters(dynamic_ebm)
  κ
  ρ
  η₂
  β
  ⋯
Observed (3): see observed(dynamic_ebm), Dict{Symbol, Any}(:φ_D => φ_D(t), :R => pop_R(t), :φ_
```

The dynamic SIR model has 5 ODEs — the standard $\theta, \varphi_I, \varphi_R, R$ plus the dormant edge state φ_D:

```
eqs = ModelingToolkit.equations(dyn_result.system)
println("Number of ODEs: ", length(eqs))
for eq in eqs
    println(eq)
end
```

```
Number of ODEs: 6
Differential(t, 1)(pop_R(t)) ~ pop_I(t)*γ
Differential(t, 1)(pop_I(t)) ~ -pop_I(t)*γ + φ_I(t)*exp((-1 + θ(t))*κ)*β*κ*(1 - ρ)
Differential(t, 1)(φ_R(t)) ~ φ_I(t)*γ - φ_R(t)*η₂ + φ_D(t)*pop_R(t)*η₁
Differential(t, 1)(φ_I(t)) ~ (φ_I(t)*exp(0)*(β + γ + η₂) - pop_I(t)*exp(0)*φ_D(t)*η₁ - φ_I(t)*
Differential(t, 1)(φ_D(t)) ~ (φ_I(t) + φ_S(t) + φ_R(t))*η₂ - φ_D(t)*η₁
Differential(t, 1)(θ(t)) ~ -φ_I(t)*β - (φ_I(t) + φ_S(t) + φ_R(t))*η₂ + φ_D(t)*η₁
```

We can also inspect the state variables and observables:

```
println("State variables: ", keys(dyn_result.variables))
println("Observables: ", keys(dyn_result.observables))
```

```
State variables: [:φ_D, :R, :φ_I, :pop_I, :pop_R, :φ_R, :θ]
Observables: [:I, :φ_S, :S, :edge_hazard, :excess_hazard]
```

Now solve the dynamic model:

```
ic_dyn = default_initial_conditions(dyn_result; seed_fraction = seed_fraction)
p_dyn = Dict(β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val, η₁ ⇒ 0.5, η₂ ⇒ 0.3)
prob_dyn = ODEProblem(dyn_result.system, merge(ic_dyn, p_dyn), tspan)
sol_dyn = solve(prob_dyn, Tsit5())
```

```
retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 24-element Vector{Float64}:
  0.0
  0.004079757551434933
  0.07662697792081173
  0.24874417320490877
  0.4868822099688729
  0.7947358810401877
  1.193604728029554
  1.6934241877653833
  2.3124910277521935
  3.0660648649896167
  
 12.698037199904364
 15.937262125148491
 20.056019151347716
 24.99648082967717
 29.5662109457882
 33.38942482511532
 36.74070296497971
 39.9303057788718
 40.0
u: 24-element Vector{Vector{Float64}}:
 [0.0, 0.001, 0.0, 0.0010000000000000009, 0.0, 1.0]
 [1.021147645543205e-6, 0.0010023656907171686, 1.0195541615679381e-6, 0.0010004635769493948
 [1.956111757932616e-5, 0.001041066761806514, 1.9009021181626078e-5, 0.0010059536131209448,
 [6.597546898864255e-5, 0.001111947156372249, 6.0404578313056954e-5, 0.001003026164994102, 0
 [0.00013422997156910761, 0.001175557271035946 3, 0.00011414364786149735, 0.0009766611934670
 [0.00022662278052511473, 0.001219738933904573 6, 0.00017714824910143782, 0.0009247647003606
 [0.00034945003087321345, 0.001238145658222271 6, 0.00024852063655237 68, 0.00084863029706708
 [0.00050367887830927 31, 0.001225487707694352 5, 0.00032394369564058614, 0.00075594165396416
 [0.00069014039571144 93, 0.001180252473455315 2, 0.00040036044598874013, 0.00065511982817753
 [0.00090555570571449 89, 0.001104187778580254 5, 0.00047483198312186 61, 0.00055473833258650
 
 [0.002449157589370533, 0.00030800855346959, 0.0008802286498811288, 0.0001147807476153688 6,
 [0.002649088102984943, 0.0001943630824190751 4, 0.0009378243133664447, 7.203374872907943e-5
 [0.002800487156716726 3, 0.0001080702671988799, 0.0009840047291813476, 3.998189333870516e-5
 [0.002896288223494312 4, 5.342109972896031e-5, 0.0010144695033491708, 1.9755456132336193e-5
 [0.002941128989436207, 2.7835505573197622e-5, 0.0010294102665900596, 1.0292742792090062e-5
 [0.002961638897502753, 1.613219060862024e-5, 0.0010365736113081795, 5.966028911666914e-6, 0
 [0.002972385049667568, 1.000212380559422e-5, 0.0010399264900658833, 3.698769385850243e-6, 0
 [0.002978793834070928 6, 6.346279928669077e-6, 0.0010417349641448686, 2.346492221016998e-6,
```

```
[0.0029789038614533815, 6.283551309905028e-6, 0.0010417408750315025, 2.3232701095990795e-6
```

```
_, I_static, _ = extract_SIR(sol_static, static_result, 5.0)
_, I_dynamic, _ = extract_SIR(sol_dyn, dyn_result, 5.0)

plot(sol_static.t, I_static, label="Static", lw=2, linestyle=:dash, color=:grey)
plot!(sol_dyn.t, I_dynamic, label="Dynamic (η₁=0.5, η₂=0.3)", lw=2, color=:red)
xlabel!("Time")
ylabel!("Fraction infected")
title!("Static vs Dynamic Network")
```

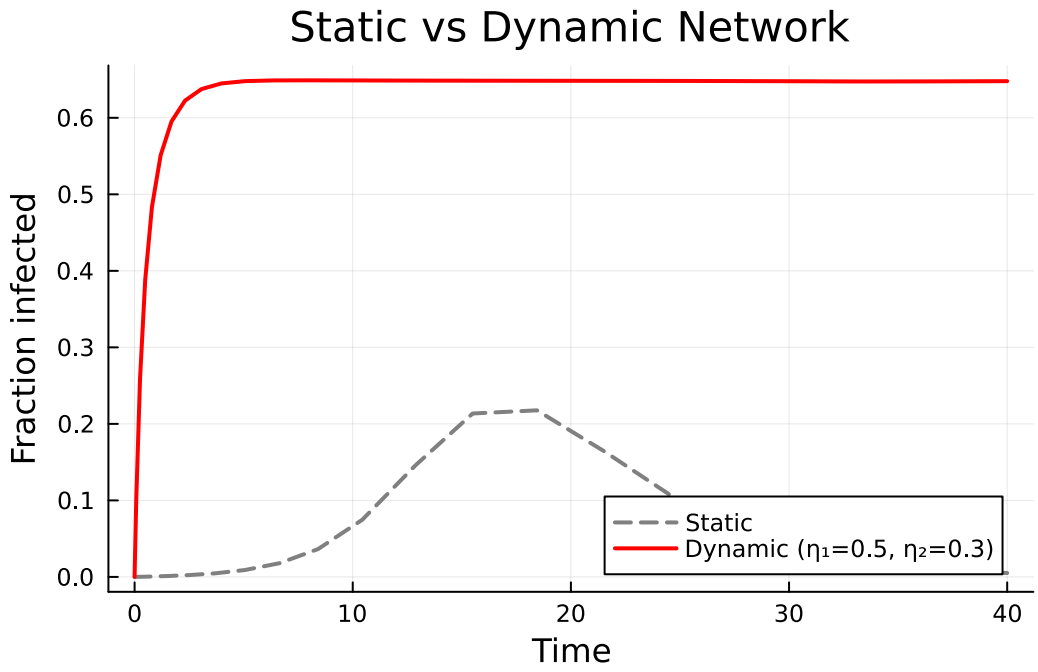

Figure 2: Comparison of the infected fraction I(t) on static versus dynamic networks.

Edge dynamics alter the epidemic because edges breaking during an epidemic can interrupt transmission chains, while new edges forming can create new pathways for infection.

Effect of Edge Dynamics

We now compare three scenarios to see how the rate of edge turnover affects the epidemic:

1. Static network — no edge dynamics ($\eta_1 = \eta_2 = 0$)

2. Slow turnover — $\eta_1 = 0.1, \eta_2 = 0.05$
3. Fast turnover — $\eta_1 = 2.0, \eta_2 = 1.2$

```
# Slow edge dynamics
p_slow = Dict(β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val, η₁ ⇒ 0.1, η₂ ⇒ 0.05)
prob_slow = ODEProblem(dyn_result.system, merge(ic_dyn, p_slow), tspan)
sol_slow = solve(prob_slow, Tsit5())

# Fast edge dynamics
p_fast = Dict(β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val, η₁ ⇒ 2.0, η₂ ⇒ 1.2)
prob_fast = ODEProblem(dyn_result.system, merge(ic_dyn, p_fast), tspan)
sol_fast = solve(prob_fast, Tsit5())
```

```
retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 59-element Vector{Float64}:
  0.0
  0.0010199404674826843
  0.01878077079178993
  0.061454888948718936
  0.12079264700789843
  0.19740257378658643
  0.2968111749460681
  0.4213685405073049
  0.5757278182219394
  0.7636357706698705
  ⋮
 33.97538025495355
 34.8357153658845
 35.69605227347944
 36.556390485954104
 37.41672937381949
 38.27706851741536
 39.1374078118266
 39.99774715869534
 40.0
u: 59-element Vector{Vector{Float64}}:
 [0.0, 0.001, 0.0, 0.0010000000000000009, 0.0, 1.0]
 [2.550606068693635e-7, 0.0010005911152102843, 2.547273360498827e-7, 0.0009991991774652431,
 [4.719369414888245e-6, 0.0010099964764410398, 4.6091484714562465e-6, 0.000984986194065371,
 [1.5592025160163824e-5, 0.0010271760159851478, 1.4477010105597571e-5, 0.000949866979998123
 [3.095352984411587e-5, 0.0010425926236032406, 2.6950239954272555e-5, 0.0009016440770969725
```

```
[5.104063375670598e-5, 0.0010537419399314072, 4.124030046389559e-5, 0.000843480574086661, 0
[7.731714283218781e-5, 0.0010596616755444983, 5.733132258400365e-5, 0.000776491907785795, 0
[0.00011032535604837597, 0.0010592825164485087, 7.449599810765176e-5, 0.000705349333158972
[0.00015108058568805692, 0.0010520013883544936, 9.239397716611572e-5, 0.000633546717679799
[0.00020017600420110492, 0.001037404858737081, 0.00011069532645609204, 0.00056479202199246
⍰
[0.0020265356606531163, 8.34819781441183e-6, 0.0007088284288833869, 2.9710040870423505e-6,
[0.0020282226893321 45, 7.359417951462525e-6, 0.0007094165149483817, 2.6191090934256924e-6,
[0.0020297099052075 03, 6.487748555455086e-6, 0.0007099349484493626, 2.30889286034295e-6, 0.
[0.0020310209729649 87, 5.719320018219827e-6, 0.0007103919784112696, 2.0354191741070916e-6,
[0.0020321767546649 96, 5.041905255417026e-6, 0.0007107948766117561, 1.794336424333909e-6, 0
[0.0020331956420615 52, 4.44472495949307e-6, 0.0007111500539254634, 1.5818082851706415e-6, 0
[0.0020340938490894 23, 3.918276192303599e-6, 0.0007114631624415621, 1.3944527042170271e-6,
[0.0020348856693422146, 3.454181492022683e-6, 0.000711739184812514, 1.2292881934481322e-6,
[0.0020348876144520253, 3.4530421070506066e-6, 0.0007117381966139245, 1.2288802472354163e-
```

```
_, I_static, _ = extract_SIR(sol_static, static_result, 5.0)
_, I_slow, _ = extract_SIR(sol_slow, dyn_result, 5.0)
_, I_fast, _ = extract_SIR(sol_fast, dyn_result, 5.0)

plot(sol_static.t, I_static, label="Static", lw=2, color=:black, linestyle=:dash)
plot!(sol_slow.t, I_slow, label="Slow (η₁=0.1, η₂=0.05)", lw=2, color=:blue)
plot!(sol_dyn.t, I_dynamic, label="Medium (η₁=0.5, η₂=0.3)", lw=2, color=:orange)
plot!(sol_fast.t, I_fast, label="Fast (η₁=2.0, η₂=1.2)", lw=2, color=:red)
xlabel!("Time")
ylabel!("Fraction infected")
title!("Effect of Edge Turnover on Epidemic Dynamics")
```

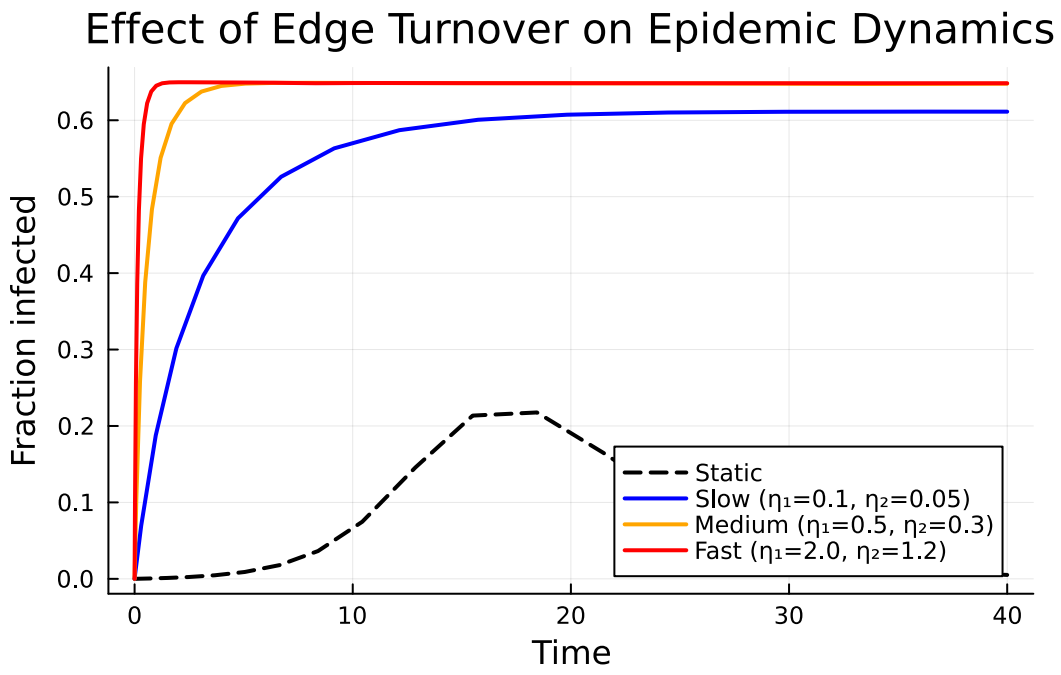

Figure 3: Effect of edge turnover rate on the epidemic curve. Faster edge dynamics alter the timing and magnitude of the peak.

The plot shows that edge dynamics modify both the peak and timing of the epidemic. The ratio η_2/η_1 controls the equilibrium fraction of dormant edges, while the absolute magnitudes of η_1 and η_2 control how quickly the network responds to population-level changes.

Inspecting Edge States

The dynamic model tracks four types of edge stubs: susceptible (φ_S), infectious (φ_I), recovered (φ_R), and dormant (φ_D). The susceptible fraction is computed algebraically as $\varphi_S = \psi'(\theta)/\psi'(1)$, while the others are ODE states.

```
κ_val = 5.0
θ_vals = compartment(sol_dyn, dyn_result, :θ)
φ_S_vals = exp.(κ_val .* (θ_vals .- 1.0))
φ_I_vals = compartment(sol_dyn, dyn_result, :φ_I)
φ_R_vals = compartment(sol_dyn, dyn_result, :φ_R)
φ_D_vals = compartment(sol_dyn, dyn_result, :φ_D)

plot(sol_dyn.t, φ_S_vals, label="φ_S (susceptible)", lw=2, color=:blue)
plot!(sol_dyn.t, φ_I_vals, label="φ_I (infectious)", lw=2, color=:red)
```

```
plot!(sol_dyn.t, φ_R_vals, label="φ_R (recovered)", lw=2, color=:green)
plot!(sol_dyn.t, φ_D_vals, label="φ_D (dormant)", lw=2, color=:purple)
xlabel!("Time")
ylabel!("Edge stub fraction")
title!("Edge State Dynamics (η₁=0.5, η₂=0.3)")
```

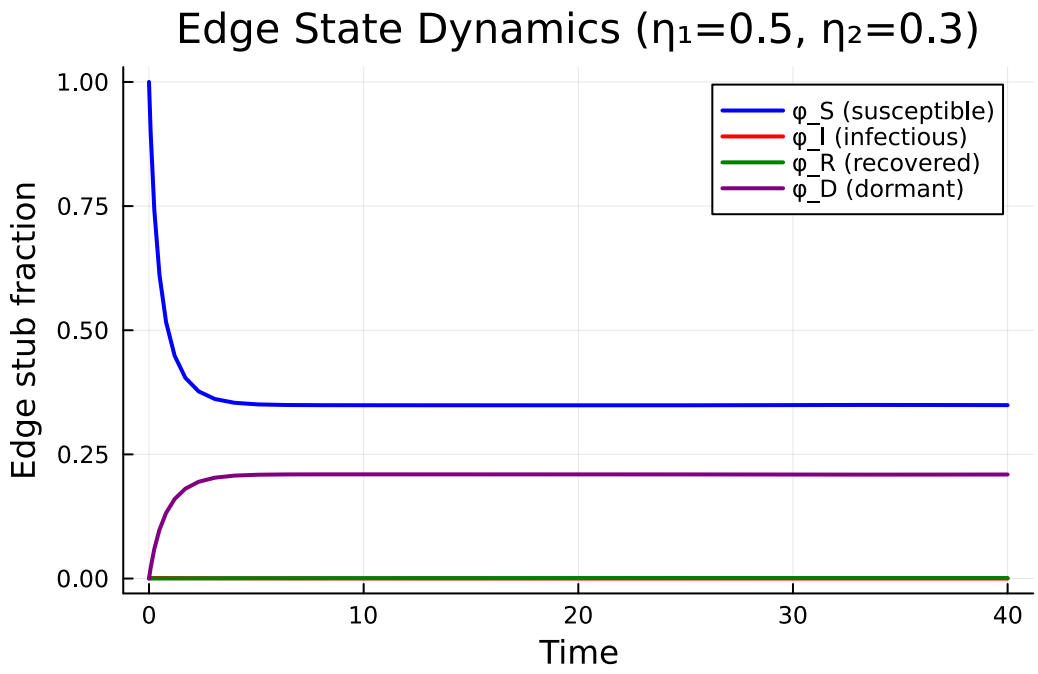

Figure 4: Edge-level state dynamics during the epidemic on a dynamic network (η_1=0.5, η_2=0.3). The dormant edge fraction (φ_D) grows as active edges break.

Initially, all edges are active and connected to susceptible neighbours ($\varphi_S \approx 1$). As the epidemic progresses, φ_S decreases while φ_I and φ_R grow. The dormant edge fraction φ_D builds up from zero as active edges break, eventually reaching an equilibrium determined by the ratio η_2/η_1.

Dynamic SEIR

The dynamic network framework also works with complex disease progressions. Here we build a dynamic SEIR model, which adds an exposed (latent) stage E that is non-infectious.

```
seir_progression = DiseaseProgression(
    [
        DiseaseStage(:E; transmission_rate=0),
```

```
        DiseaseStage(:I; transmission_rate=β),
        DiseaseStage(:R)
    ],
    [
        DiseaseTransition(:E, :I, σ),
        DiseaseTransition(:I, :R, γ)
    ];
    entry=:E
)
dyn_seir_model = DynamicConfigurationModel(pgf, seir_progression, η₁, η₂)
dyn_seir_result = build_edge_system(dyn_seir_model)
```

```
EdgeModelSystem(Model dynamic_ebm:
Equations (8):
  8 standard: see equations(dynamic_ebm)
Unknowns (8): see unknowns(dynamic_ebm)
  pop_R(t)
  pop_I(t)
  pop_E(t)
  φ_R(t)
  □
Parameters (7): see parameters(dynamic_ebm)
  κ
  ρ
  η₂
  β
  □
Observed (3): see observed(dynamic_ebm), Dict{Symbol, Any}(:φ_D => φ_D(t), :R => pop_R(t), :φ_
```

The dynamic SEIR model has 6 ODEs — adding φ_E to the dynamic SIR's set:

```
seir_eqs = ModelingToolkit.equations(dyn_seir_result.system)
println("Number of ODEs: ", length(seir_eqs))
for eq in seir_eqs
    println(eq)
end
```

```
Number of ODEs: 8
Differential(t, 1)(pop_R(t)) ~ pop_I(t)*γ
Differential(t, 1)(pop_I(t)) ~ -pop_I(t)*γ + pop_E(t)*σ
Differential(t, 1)(pop_E(t)) ~ -pop_E(t)*σ + φ_I(t)*exp((-1 + θ(t))*κ)*β*κ*(1 - ρ)
```

```
Differential(t, 1)(φ_R(t)) ~ φ_I(t)*γ - φ_R(t)*η₂ + φ_D(t)*pop_R(t)*η₁
Differential(t, 1)(φ_I(t)) ~ -φ_I(t)*(β + γ + η₂) + φ_E(t)*σ + pop_I(t)*φ_D(t)*η₁
Differential(t, 1)(φ_E(t)) ~ (φ_E(t)*exp(0)*(η₂ + σ) - pop_E(t)*exp(0)*φ_D(t)*η₁ - φ_I(t)*exp
Differential(t, 1)(φ_D(t)) ~ (φ_I(t) + φ_E(t) + φ_S(t) + φ_R(t))*η₂ - φ_D(t)*η₁
Differential(t, 1)(θ(t)) ~ -φ_I(t)*β - (φ_I(t) + φ_E(t) + φ_S(t) + φ_R(t))*η₂ + φ_D(t)*η₁
```

```
println("State variables: ", keys(dyn_seir_result.variables))
```

```
State variables: [:φ_D, :R, :φ_I, :pop_I, :pop_R, :φ_R, :θ, :φ_E, :pop_E]
```

The dynamic framework is fully composable — any disease progression supported by `EdgeBasedModels.jl` can be combined with edge formation and breaking dynamics, simply by wrapping it in a `DynamicConfigurationModel` instead of a `StaticConfigurationModel`.

Simulation validation

This vignette focuses on a scenario for which `NetworkOutbreaks.jl` does not yet provide an out-of-the-box stochastic ground truth (multi-type host graphs with prescribed mixing matrices, time-varying networks via the EBCM rewiring schedule, multiplex layers, degree-correlated configuration models, clustered graphs with prescribed triangle counts, or final-size sweeps over R_0).

A future revision will add the missing primitives to NetworkOutbreaks.jl and overlay the corresponding Gillespie SSA ribbon here. Until then, see vignette 01 for the validation pattern on a single-layer Poisson configuration-model network with the same canonical parameters.